

1. Basic and Fundamental of FT
 2. we have explored the huggingface in detail
- these only point we have covered in our last 3 classes

today- non instruction FT- HF then Unsloth

tomorrow - instruction FT

next Saturday- Preference tuning

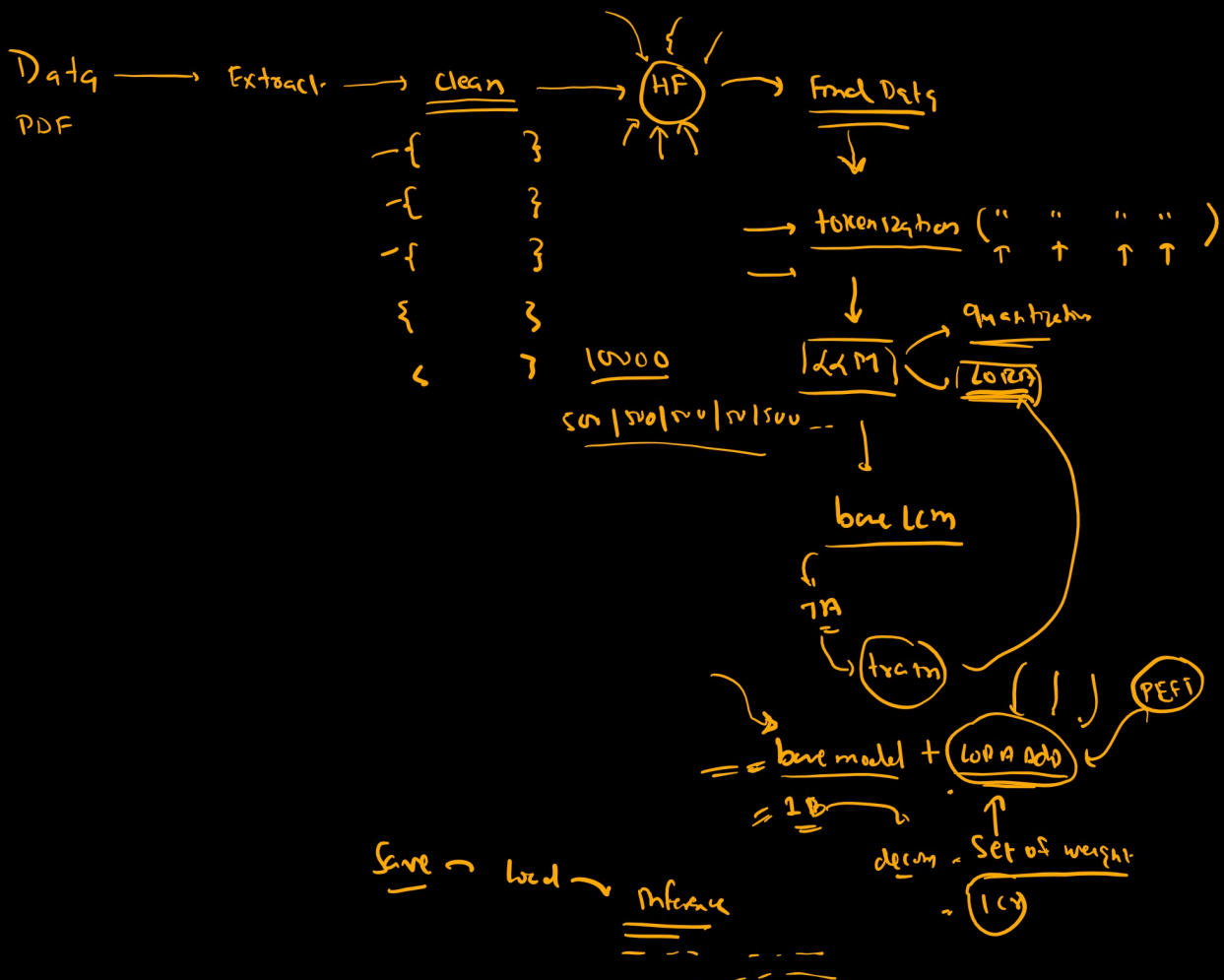
next Sunday- LORA, and will discussion about the Quantization

One Mega Assignment i will give you after the Finetuning

complete this FT

after this RAG

I am sunny 0 0 0
i am sunny savita 0 0
i am sunny savita AI 0
i am sunny savita AI mentor



pharama --> pdf --> retrain or finetune my own model

1. Non-instruction FT

whenever we have a RAW data

pdf, docx, ppt, text

2. Instruction FT

input <-> output

3. Preference tuning

choose<->rejected

Question: In real time how we get a raw data for non instruction Finetuning?

SharePoint

Confluence

Google Drive

AWS S3

Azure Blob Storage

Internal document portal

Download as PDF / DOCX / HTML / TXT / Markdown

Ques: In real time how we get a data for instruction finetuning?

Method 1: Human-created Q&A data

Domain experts manually create questions and answers.

Example:

```
{
  "instruction": "Explain the mechanism of action of Metformin.",
  "input": "",
  "output": "Metformin primarily activates AMPK, reduces hepatic gluconeogenesis, and improves glucose uptake."
}
```

Method 2: Use existing customer/support conversations

Companies may already have real conversations:

Customer question

Doctor/agent/expert answer

Support ticket

Chat transcript

Email thread

Medical information query

These can be cleaned and converted into instruction format.

Example:

```
{
  "instruction": "What are common side effects of Metformin?",
  "input": "",
  "output": "Common side effects include nausea, abdominal discomfort, metallic taste, and diarrhea, especially during treatment initiation or dose escalation."
}
```

Method 3: Synthetic data generation

We can use a strong LLM to generate Q&A pairs from raw documents.

Flow:

Raw pharma document

↓

LLM generates questions and answers

↓

Human/domain expert reviews

↓

Clean instruction dataset

↓

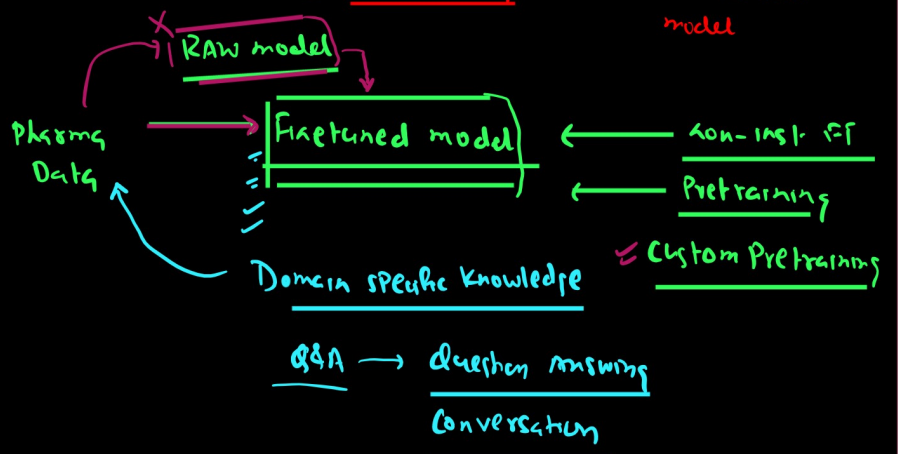
Instruction fine-tuning

Important: In regulated domains like pharma or healthcare, synthetic data should be reviewed by domain experts.

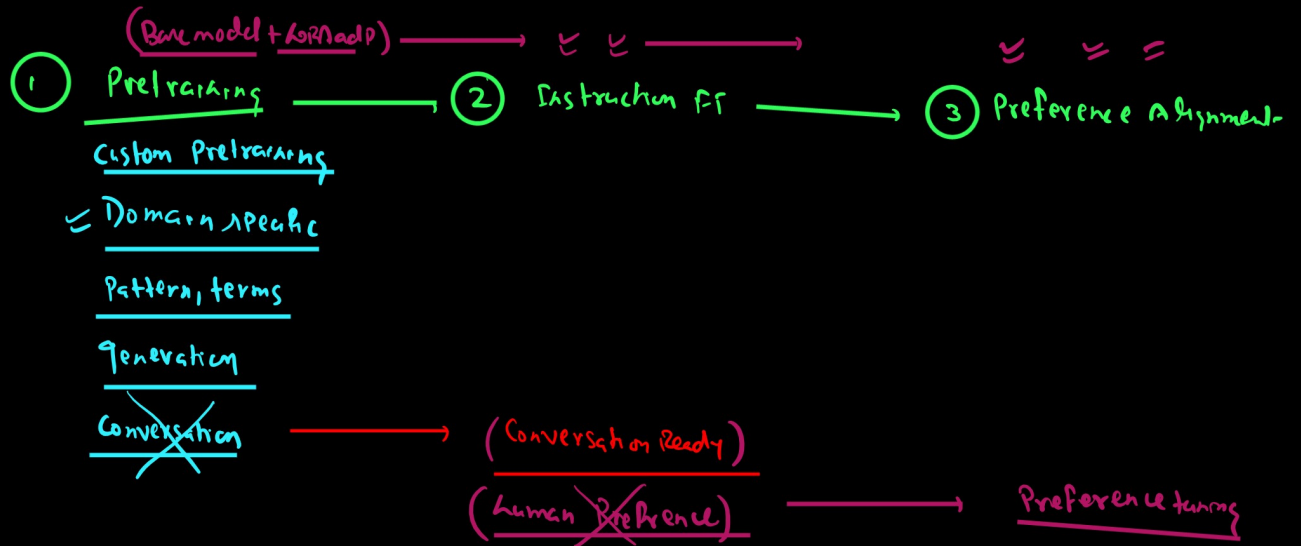
For instruction fine-tuning, we need structured examples where each record has a question or instruction and an ideal answer. This data can be written by experts, extracted from real conversations, or generated from documents and reviewed. Instruction fine-tuning teaches the model how to respond to users, not just how the domain text looks.

PDF text
RAW Data → FT domain
Non instruction FT

- ① RAW Data ← Pharmg
- ② Read the Data (Data Parsing)
- ③ Cleaning of the data
- ④ Final Data
- ⑤ hf compatible format
- ⑥ Pre Processing (tokenization)
- ⑦ Loaded the model Quantize model
LORA Adapter
- ⑧ FT of the model (PEFT)
Full ~~Model~~
LoRA → Subset of Parameter
- ⑨ Save / Push Hugging face hub
- ⑩ Inferencing → Prediction with model



LLM training



len1 $\rightarrow$ my name is sunny $\leftarrow$ 4

len2 $\rightarrow$ my friend name is yash and Rahul $\leftarrow$ 7

len3 $\rightarrow$ I am DS & AI eng and my friend is dev $\leftarrow$ 11

MAX-LENGTH

$\downarrow$

PADDING <EOS> <S>

my name is sunny <S> <S> <S> <S> ... <S>

$\rightarrow$ < > < > < > < >

len1 --- 413

len2 --- 512

len3 --- 1000

len4 --- 300

len5 --- 650

} PADDING $\rightarrow$ 1000

--- 1000

--- 1000

--- 1000

--- 1000

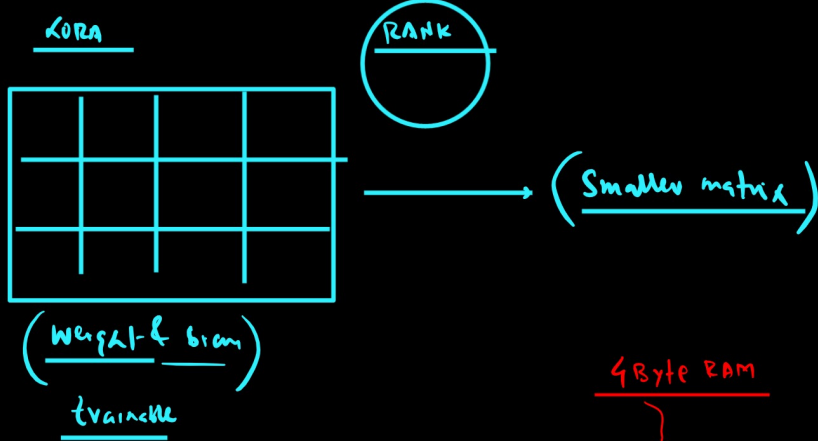
--- 1000

} max-token $\rightarrow$ 512

truncate

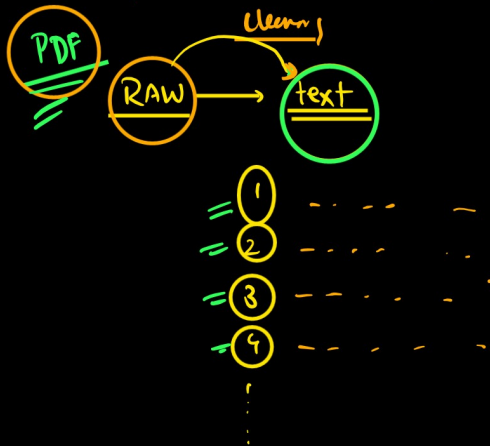
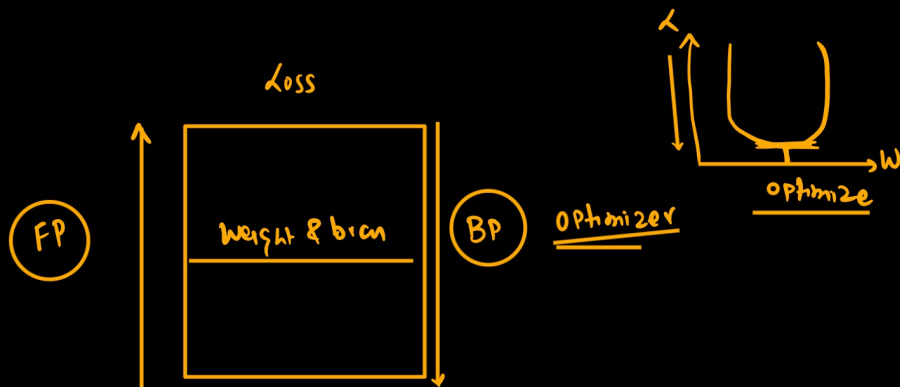
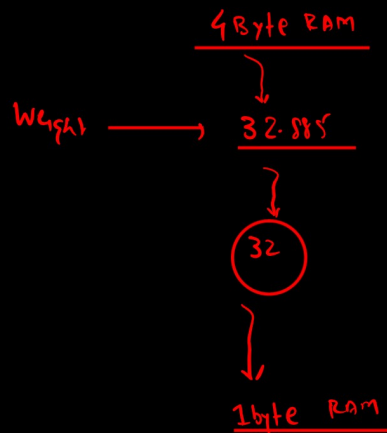
512 $\rightarrow$ block

token



Quantizing

Float → 32 bit
int → 8 bit
AP → 16 bit



Paragraph

shift = 1

You need to change your setting to get notification.

IIP

You need to need to change to cheng your change your setting

OIP

change your setting to

next token prediction



Instruction

JSONL | CSV

manually

{
 instruction: Summarize the given input,
 input: _____
 output: _____
}

Format

⇒ text: "instruction# ---- input# ---- output# ----"

Alpaca

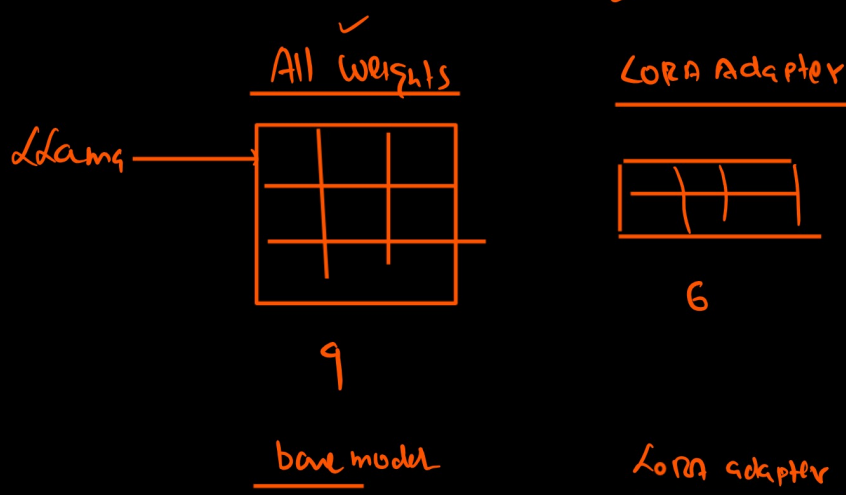
Well know

{ instruction: Summarize the OIP
IIP : "Sunny in a AI mentor"
OIP : "Sunny mentor" } SFT

{ instruction: "what is a normal diabetes value"
IIP : " "
OIP : "Fasting <100 >120" }

Chapter

{ USER
System }



① = base model + LoRA adapter

```
from peft import PeftModel  
inference_model = PeftModel.from_pretrained(inference_base_model, config.adapter_dir)
```

② = base model + (LoRA adapter) Permanently

```
# Load the trained LoRA adapter on top of the base model.  
model_with_adapter = PeftModel.from_pretrained(  
    base_model,
```

```
# Merge LoRA adapter weights into the base model weights.  
merged_model = model_with_adapter.merge_and_unload()
```

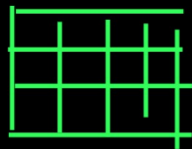
Base model

← (FT)

data

1B Param

~~Full Fine-tune~~



Rank



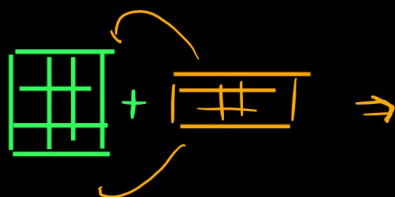
1M



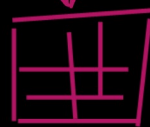
≡

Non inst data

≡ (LoRA Adapter 1)



Non inst data



inst tuning =



(LoRA adapter 2)

FT → ① Non-instruction tuning ← RAW data

② Instruction tuning (SFT) ← instruction data

(APIs → inst | IP | OP)

open AI → System | User | AI

Format → ✓ ✓ ✓

① manual way

② Synthetic data (data using AI)

③ Preference tuning ← Align the model with human responses

Prompt

Q. how to prepare for the exam with in 45 hr.?

Rejected

Ans1 → Study 45 hr continuously without break.

Selected

Ans2 → take a imp question from previous ques paper take a short a bit study

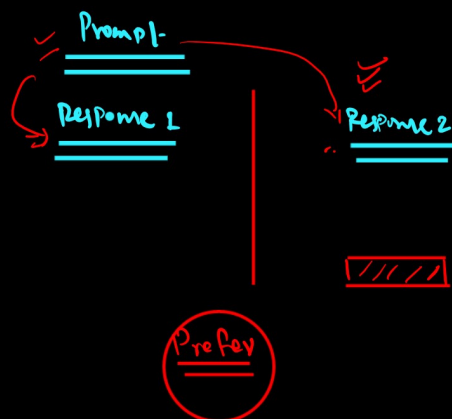
Prompt

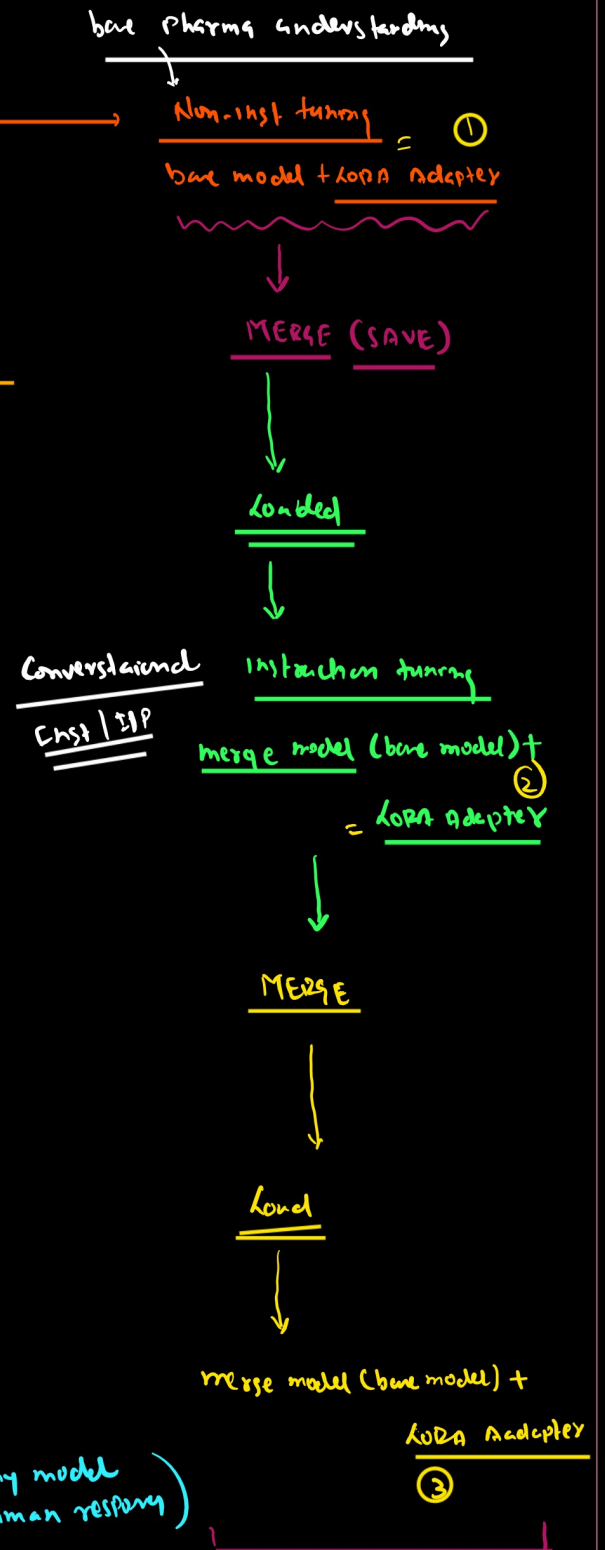
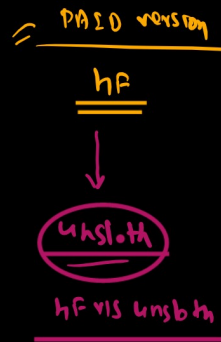
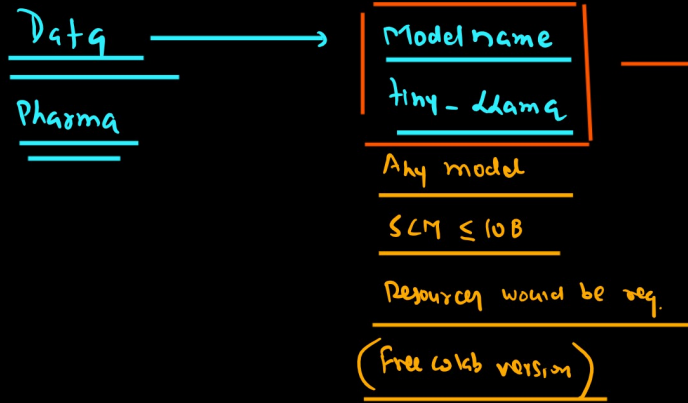
Chosen

Rejected

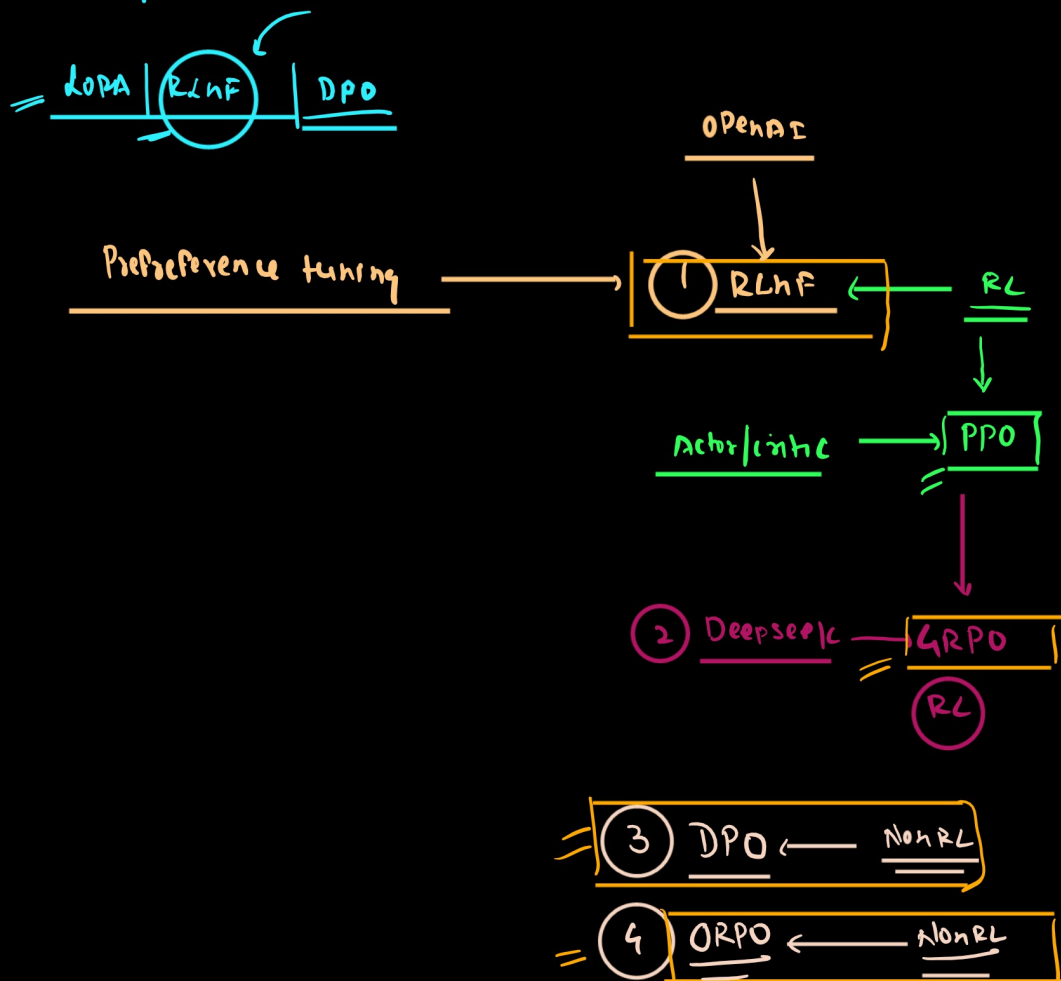
2 way → ① manual way ← SME low | low

② with the help of AI ← RAW Inst } Data



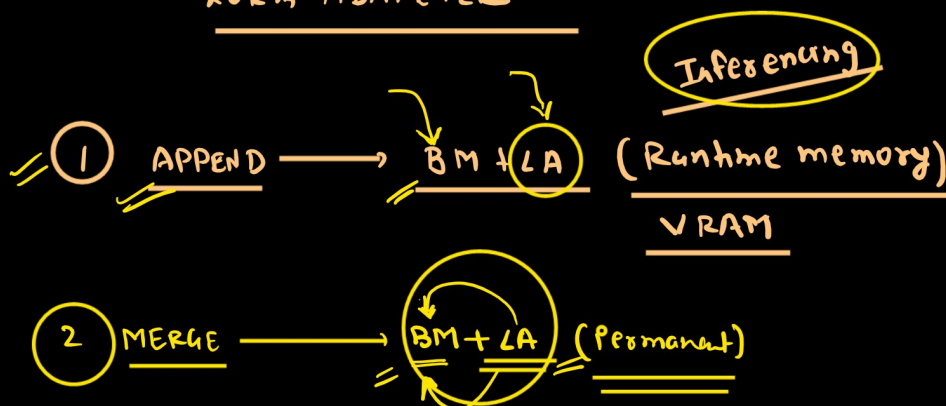


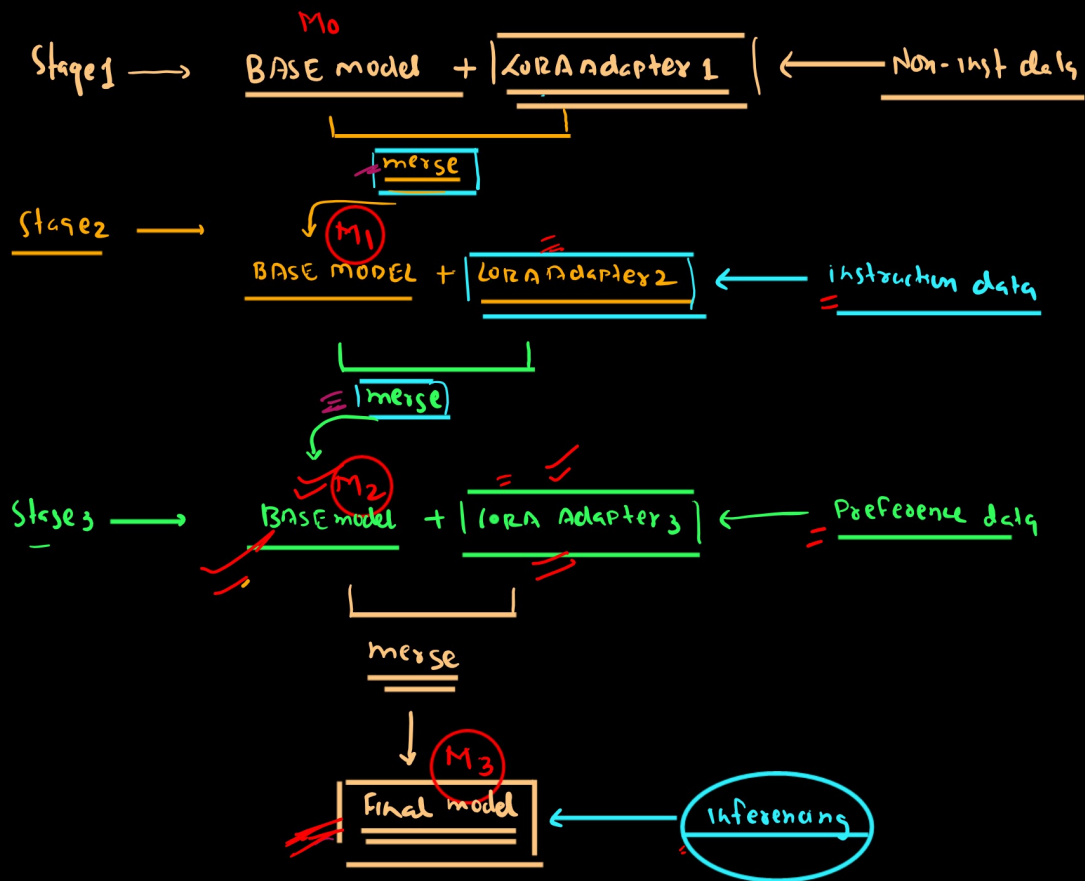
{"prompt": "### Instruction:\nExplain the primary mechanism of action of metformin.\n\n### Response:\n",
 "chosen": "Metformin primarily acts by activating AMP-activated protein kinase, also called AMPK. AMPK is a central metabolic regulator that promotes glucose uptake and fatty acid oxidation while reducing hepatic gluconeogenesis, which helps lower blood glucose levels.",
 "rejected": "Metformin mainly works by increasing insulin secretion from the pancreas, and kidney function is usually not very relevant. Its side effects are generally not important unless the patient feels very sick.",
 "source_page": 1, "topic": "Metformin pharmacology"}



BASE MODEL

LORA ADAPTER

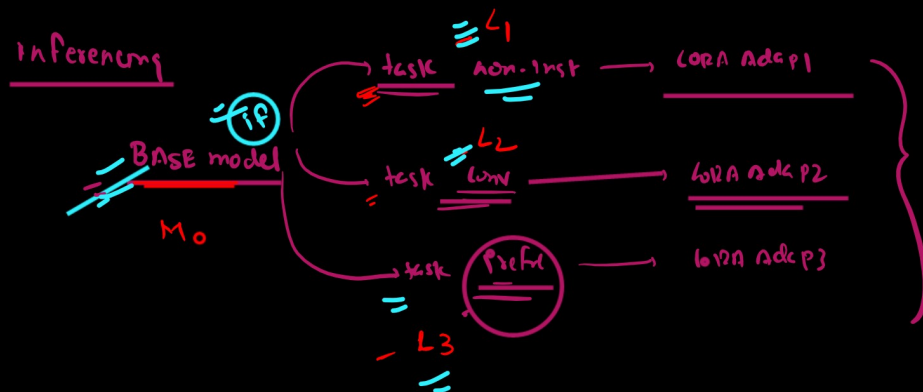
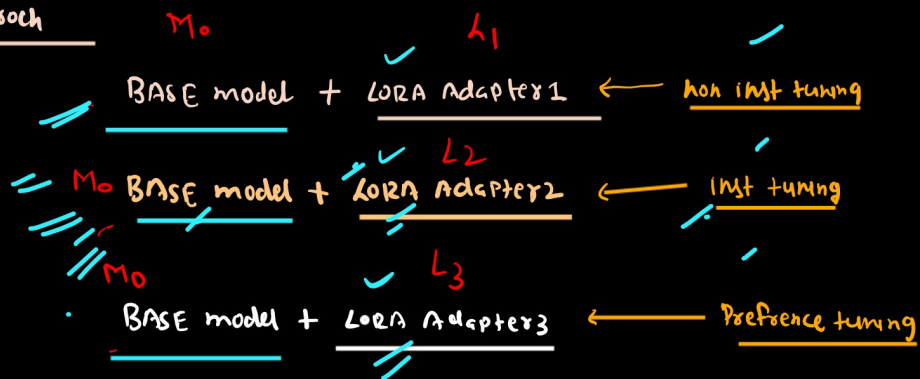




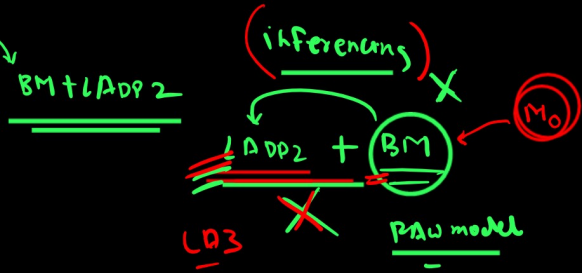
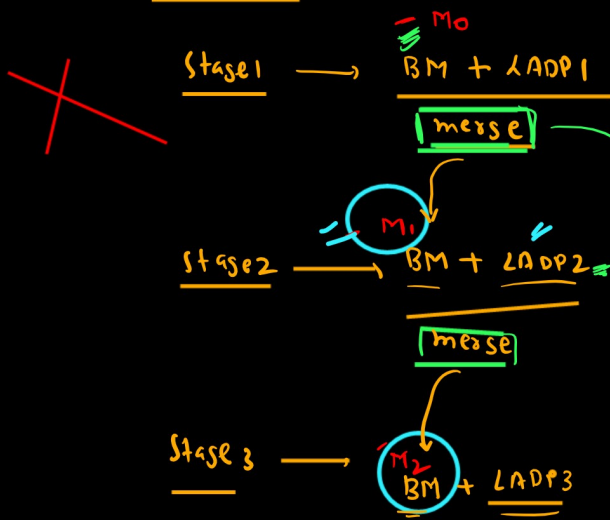
if you want one single final model

then take this approach

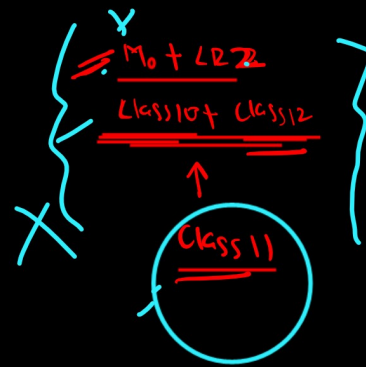
second approach



Confusion



$M_0 = \text{RAW model} \leftarrow \text{Class 10 Student}$
 $M_1 = \text{merge (Stage 1)} \leftarrow \text{Class 10 + Class 11}$
 $M_2 = \text{merge (Stage 2)}$
 $L_2 (Class 12)$
 L_1



- ① KLM training pipeline
- ② Stg1,2,3 ←
- ③ HF

OpenSource
Weight / Locality / param

- ① KORA | QORA
- ② ELHF | DPO ← to train
- ③ UNSLOTH
- ④ OPENAI API ← 4PT - 4,5 - ...

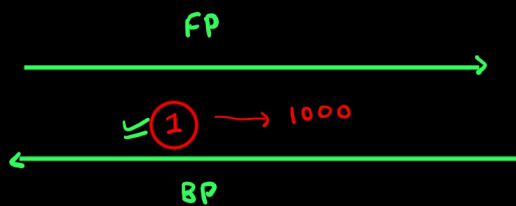
2-clony

Sunday → RAH

1 epoch = 100 steps

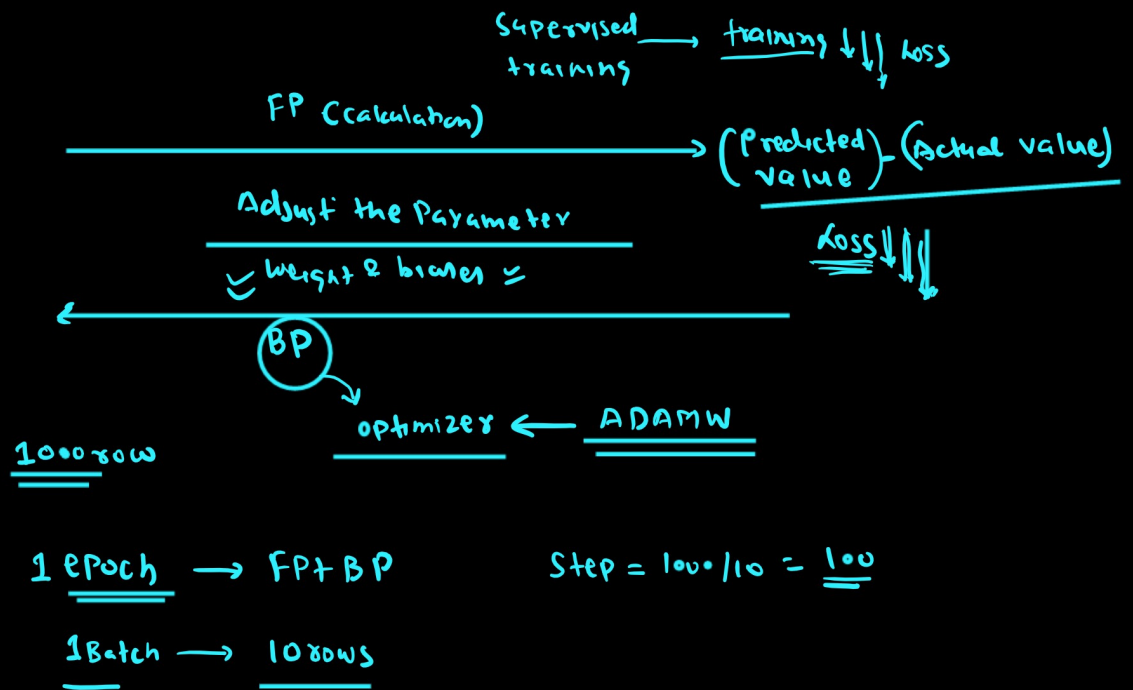
Data = (1000) example

Batch-size = 10 epoch-size = 1

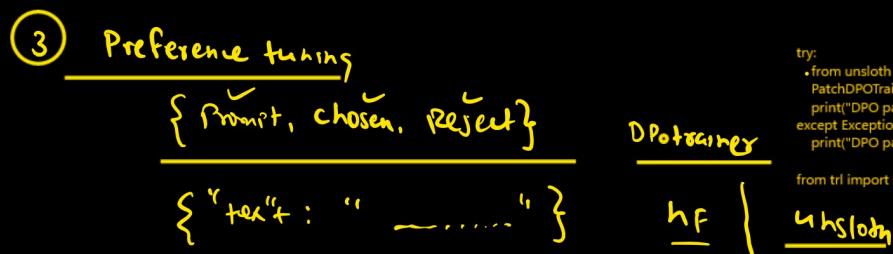
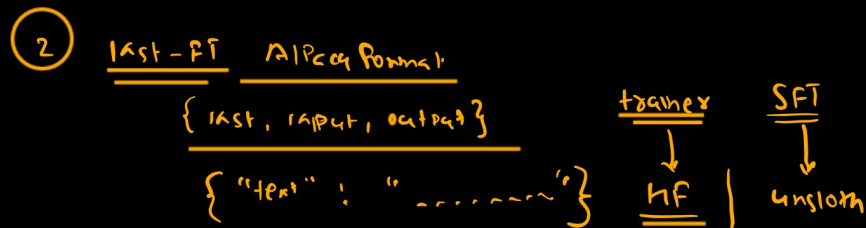
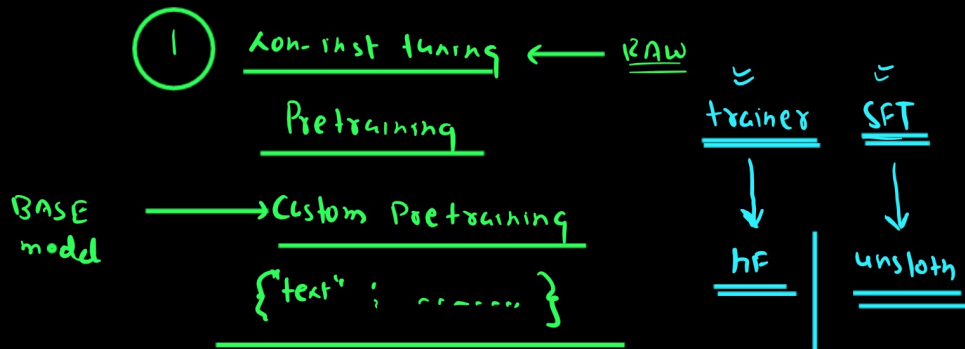


$$\frac{1000}{10} = 100$$

(1 step = 10 example)

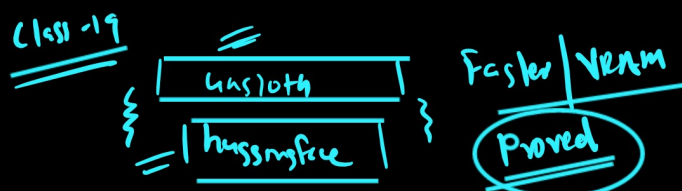


LLM-training



```
try:
    from unsloth import PatchDPOTrainer
    PatchDPOTrainer()
    print("DPO patch applied.")
except Exception as e:
    print("DPO patch skipped:", repr(e))
```

```
from trl import DPOTrainer
```



```
try:
    from unsloth import PatchDPOTrainer
    PatchDPOTrainer()
    print("DPO patch applied.")
except Exception as e:
    print("DPO patch skipped:", repr(e))
from trl import DPOTrainer
```

Unsloth patch applied

↓

TRL DPOTrainer imported

↓

DPO training runs with Unsloth optimization

OpenAI

1. vision
 2. embedding
 3. theoretical concept
- within a session

RAG